클라우드 데이터베이스

Cloud Database

클라우드 데이터베이스 Cloud Database

10주차
Architecture

클라우드 데이터베이스 Cloud Database

Today's Goal

학습 목표

- 데이터베이스의 아키텍쳐와 SQL 처리 과정을 이해한다.
- PostgreSQL 의 구조를 이해하여 데이터베이스를 효율적으로 운영한다.

Today's Contents

학습 내용

- PostgreSQL 의 5단계 각각에 대한 구성과 역할을 하나씩 살펴본다.
- SQL구문 수행시 5단계의 과정을 하나씩 살펴본다.



클라우드 데이터베이스 Cloud Database

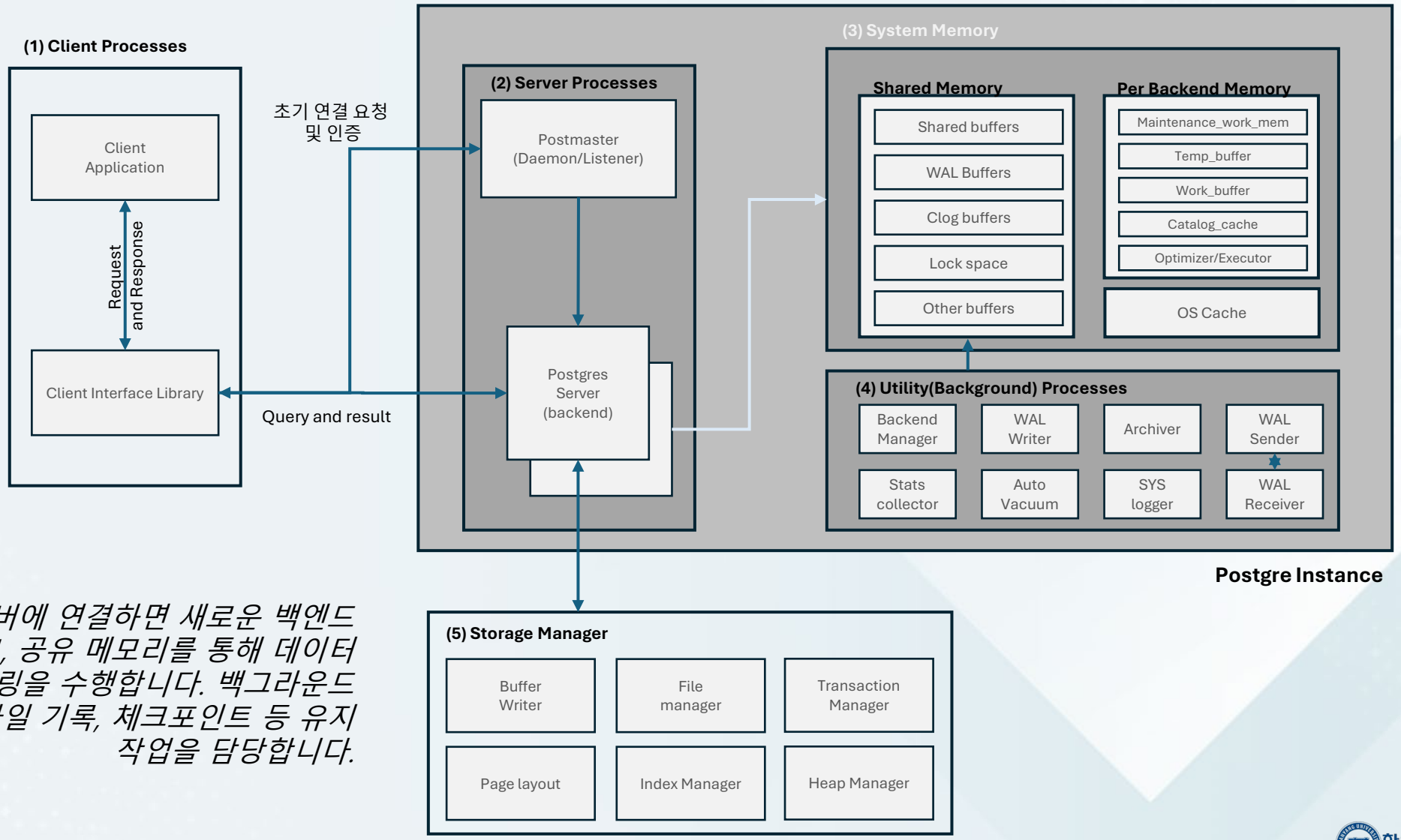
PostgreSQL 아키텍처

데이터베이스의 아키텍쳐 이해

- 현업에서 수백, 수천만 건 이상의 데이터가 저장되는 데이터베이스에서 다양한 조회를 빠르게 수행하기 위해서는 데이터베이스의 아키텍쳐와 SQL 처리 과정, 인덱스와 조인의 수행원리 등을 이해하는 것이 중요함.

- PostgreSQL의 구조를 이해하는 것은 자동차를 잘 운영하고 적합한 성능을 내기 이해 필요한 과정임

PostgreSQL 아키텍쳐



클라이언트가 서버에 연결하면 새로운 백엔드 프로세스가 생성되고, 공유 메모리를 통해 데이터 캐싱 및 로그 버퍼링을 수행합니다. 백그라운드 프로세스들은 WAL 파일 기록, 체크포인트 등 유지 작업을 담당합니다.

1. Client Processes

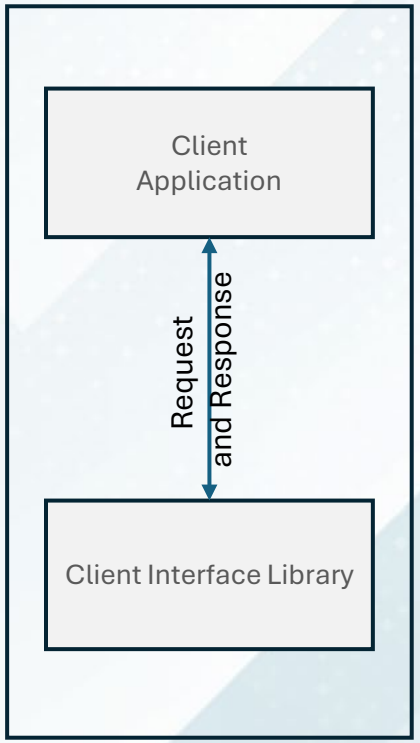
개념:

- 클라이언트 프로세스는 데이터베이스에 질의를 보내는 프런트엔드 프로그램을 의미함
- 예를 들어, psql 도구나 웹 애플리케이션의 DB 드라이버가 이에 해당하며, PostgreSQL은 각 프로세스가 1명의 사용자 모델로 작동하여 새로운 연결이 열릴 때마다 서버에 해당 클라이언트를 위한 프로세스가 생성됨

설명:

- 클라이언트는 일반적으로 SQL 질의 문자열을 서버에 전송하고 결과를 수신하는 역할을 함.
- 클라이언트 프로세스는 DB 엔진의 일부가 아닌, DB 서버에 요청을 전달하는 기능을 수행함.
- 예를 들어, 사용자가 웹 애플리케이션에서 "회원 목록 조회" 버튼을 클릭하면, 클라이언트가 PostgreSQL 서버로 SELECT 질의를 보내고 결과를 표시함.

(1) Client Processes



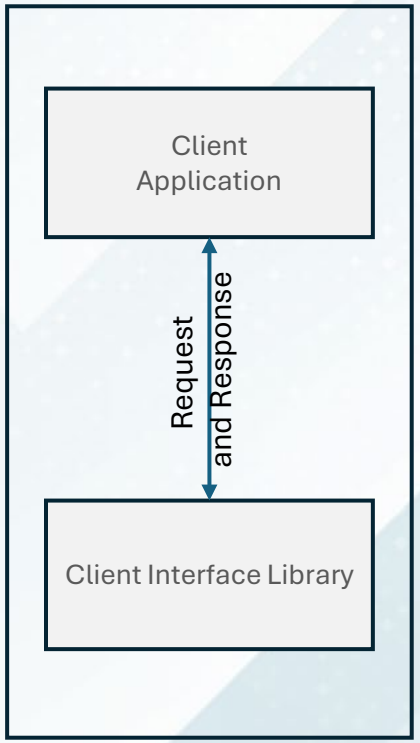
1. Client Processes

비유:

- 클라이언트를 식당의 손님에 비유하면, DB 서버는 주방이고, 클라이언트 프로세스는 손님의 주문을 접수해 주방에 전달하는 웨이터와 같음.
- 클라이언트는 주문을 전달하고 결과를 기다리는 것과 유사함

라이브러리	설명
Libpg	저수준(low-level) 인터페이스를 제공하는 C 기반 라이브러리
JDBC	Java Database Connectivity.
ODBC	Open Database Connectivity
Libpqxx	Libpg 대비 고수준 인터페이스를 제공하는 C++ 라이브러리
pyscopg	Pythonic 인터페이스를 제공하는 파이썬 라이브러리

(1) Client Processes



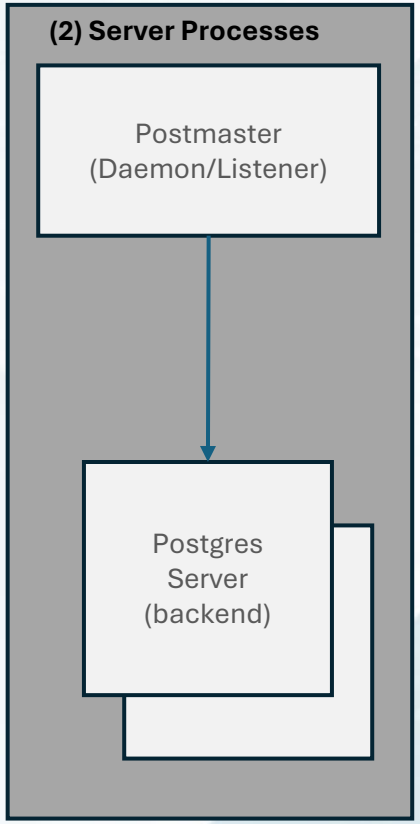
2. Server Processes

Postmaster:

- Postgres Server 의 핵심 프로세스로, 연결 수락, 백엔드 관리, 공유 메모리 및 시스템 자원 관리를 포함한 전체 시스템 관리를 담당함.
- PostgreSQL서버가 시작될 때 Postmaster가 가장 먼저 실행되며, 이후 시스템에 필요한 모든 프로세스를 생성하고 관리함.

Postgres Server(백엔드):

- 클라이언트 애플리케이션의 연결을 처리하고 SQL 문을 실제로 실행하는 작업자 프로세스로, SQL 문 실행, 트랜잭션 관리 및 결과 반환을 수행함.



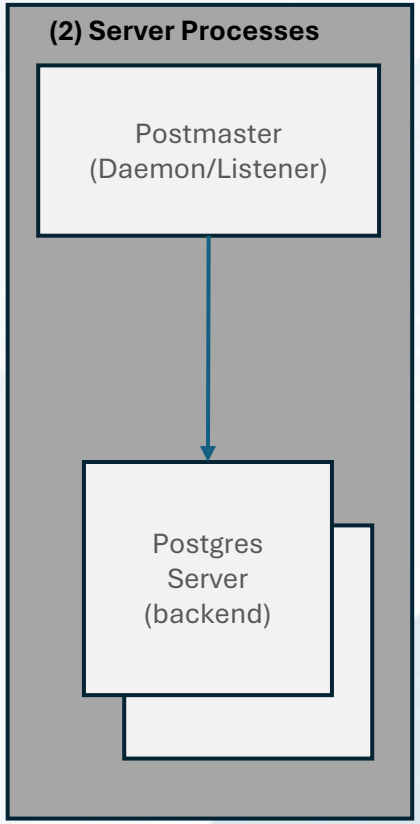
2. Server Processes

특징:

- PostgreSQL 은 멀티프로세스 아키텍처를 채택하여 각 연결마다 독립적인 프로세스를 운영함으로써 격리를 강화함. 하지만 프로세스 수가 증가하면 자원 소비가 늘어나므로, max_connections 매개변수를 통해 동시 연결 수를 제한할 수 있음

비유:

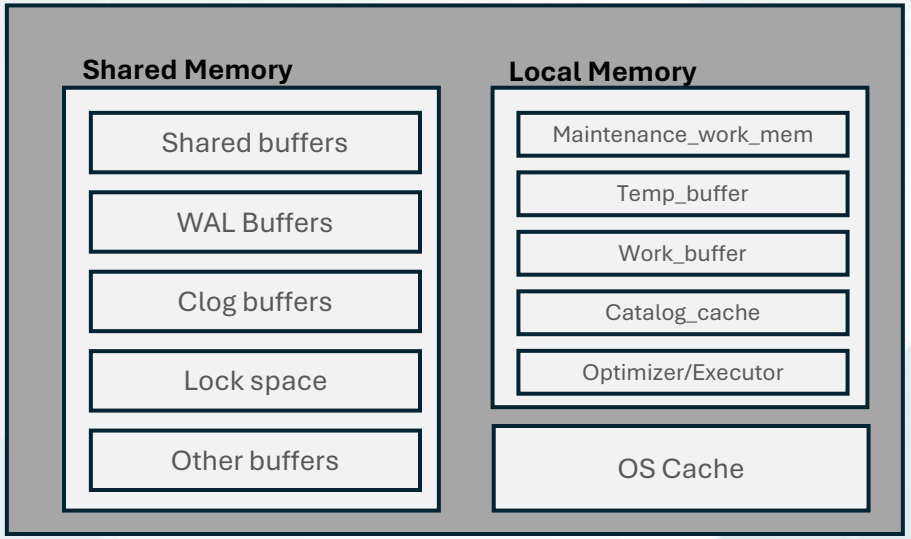
- 서버 프로세스는 식당 주방에 비유되며, Postmaster는 주방장 역할을 하여 손님이 올 때마다 새로운 요리사(백엔드)를 배정하는 것처럼, 각 요리사는 손님의 주문을 처리하고 요리가 끝날 때까지 대기하며, 주방장(Postmaster)은 유지보수와 프로세스 관리를 담당함



3. System Processes

개념:

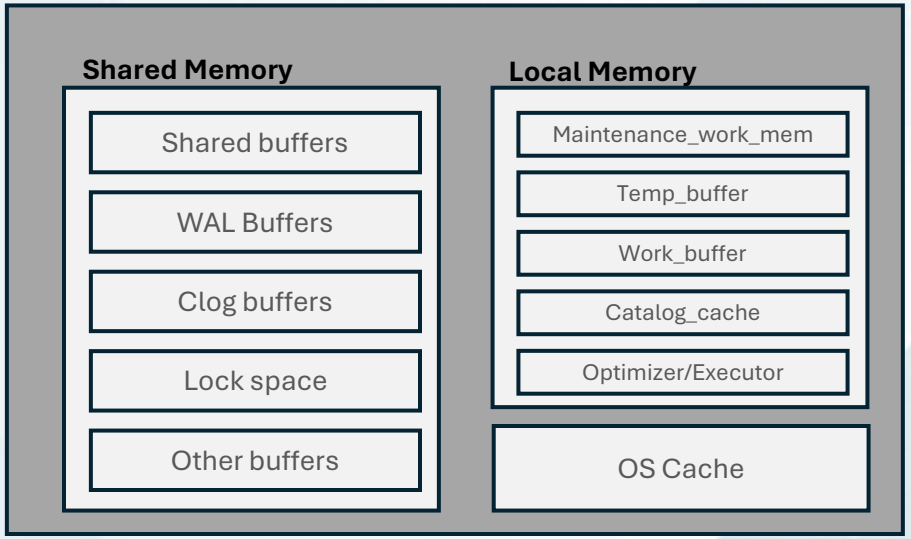
- PostgreSQL서버는 성능 향상을 위해 두 가지 메모리 공간, 즉 여러 프로세스가 공유하는 공유 메모리(Shared Memory)와 각 백엔드 프로세스별로 할당된 지역 메모리(Local Memory)를 사용함.



3. System Processes

공유 메모리:

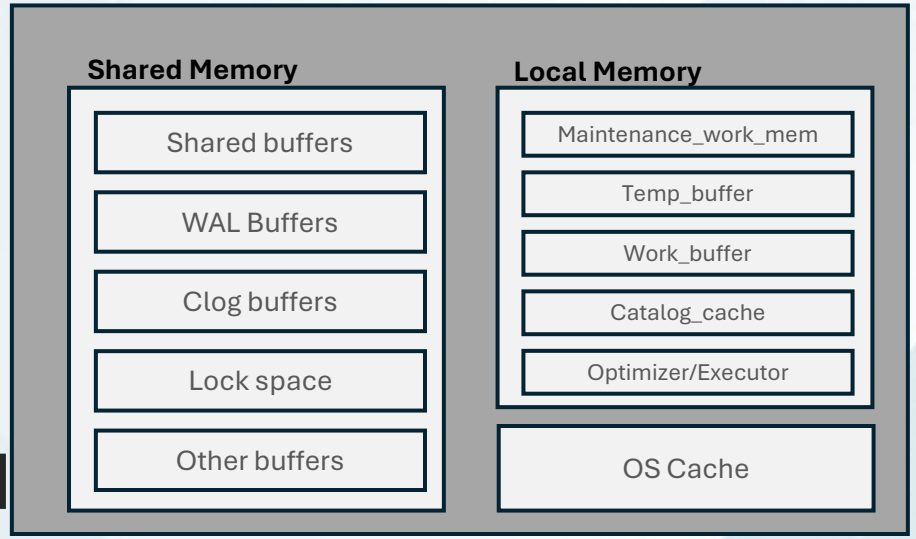
- 서버 프로세스는 데이터베이스 운영에 필요한 데이터를 공유 버퍼(Shared Buffer)에 저장하여 활용함.
- 이 버퍼는 디스크의 테이블이나 인덱스 페이지를 캐싱해 디스크 I/O를 줄이고 성능을 향상시킴
- 자주 사용되는 데이터는 이 메모리에 남아 재사용되며, 변경된 데이터는 일정 시간이 지나면 디스크에 기록됨.
- 또한, 트랜잭션 로그(WAL) 기록을 위한 WAL 버퍼도 포함되어 있으며, 이는 시스템 장애 시 복구를 위해 변경 기록을 일시적으로 저장함.



3. System Processes

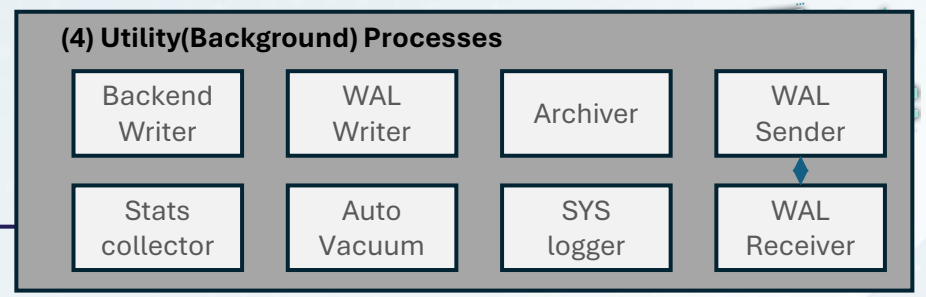
지역 메모리:

- 각 백엔드 프로세스는 자신의 로컬 메모리 영역을 가지고 있음.
- 이 메모리는 정렬, 해시 연산 및 쿼리 실행 중의 임시 작업에 사용됨.
- 예를 들어, work_mem 파라미터에 의해 설정되는 정렬/해시 메모리와 temp_buffers로 지정되는 임시 테이블 저장 메모리가 있음.
- 백엔드 프로세스는 이러한 지역 메모리를 사용하여 개별 쿼리를 처리하며, 이 메모리는 다른 프로세스와 공유되지 않으므로 세션 전용 작업 공간으로 간주됨.



학습내용

4. Utility Processes



정의:

- 데이터베이스의 일관성과 성능을 유지하기 위해 여러 관리 작업을 수행하는 백그라운드 프로세스임.

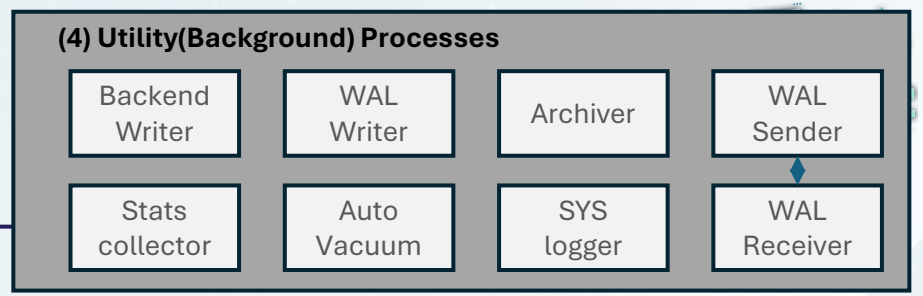
설명:

- 이들 프로세스는 사용자 요청을 직접 처리하지 않지만, 데이터 무결성과 성능 유지를 위한 필수 지원 역할을 함.
- 예를 들어, WAL writer나 체크포인터가 없으면 메인 프로세스가 모든 트랜잭션 로그를 처리해야 하므로 응답 속도가 저하됨. PostgreSQL은 이러한 백엔드 지원 덕분에 안정적으로 운영되는 것임.

비유:

- 백그라운드 프로세스는 극장의 무대 뒤 스태프에 비유할 수 있음.
  무대에서 배우가 공연을 하는 동안, 조명, 무대 전환, 데이터 복구 등을 담당하는 스태프가 보이지 않게 지원하여 공연이 원활하게 진행되도록 함.

4. Utility Processes



주요 백그라운드 프로세스들:

- Sys Logger에러 메시지와 경고를 로그 파일에 기록.
- Background Writer: 공유 버퍼의 더티(Dirty) 페이지를 주기적으로 디스크에 기록하여 I/O 부하를 줄여 줌.
- WAL Writer: WA버퍼의 내용을 WAL 로그 파일에 지속적으로 기록하여 데이터 변경의 내구성을 보장함.
- Auto vacuum: 테이블 및 인덱스에서 죽은 행을 자동으로 정리하고, 테이블과 인덱스가 적절하게 유지되도록 관리하는 역할 수행
- Archiver: WAL 로그 파일을 지정된 장소로 복사하여 백업 및 PITR을 위해 보관.
- Stats Collector: D화용 통계 정보를 수집하여 관리자가 모니터링할 수 있도록 함.
- WAL Sender & WAL Receiver:와복제와 고가용성을 위한 구성 요소.
- Check pointer: 정해진 시점에 공유 버퍼의 변경 내용을 디스크에 기록하여 메모리와 디스크 내용을 동기화 함.

4. Utility Processes

WAL (Write-Ahead Log)

- 데이터가 데이터베이스에 기록 전에 별도의 로그 파일에 데이터 변경 사항을 기록하고, 이를 통해 충돌 또는 기타 예기치 못한 중단 발생 시 데이터를 복구해 데이터가 일관성을 유지하고 트랜잭션이 ACID가 되도록 보장함.

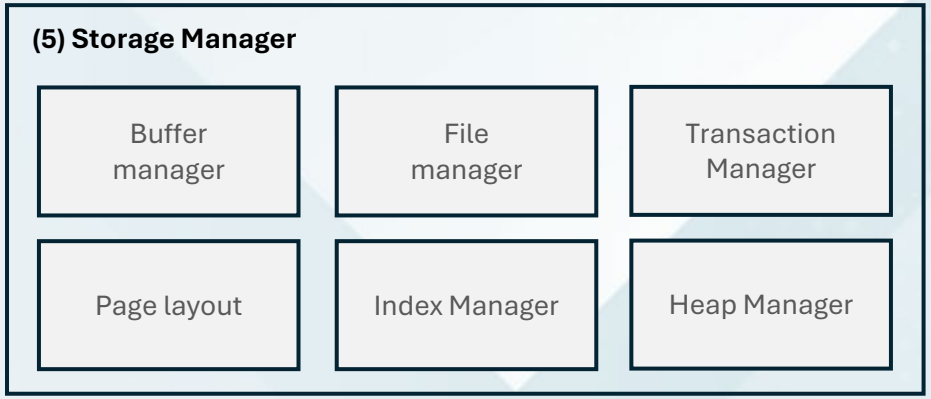
- WAL 은 WAL 파일 끝에 새 로그 항목이 기록되고, 파일이 가득차면 이전 로그 항목을 덮어쓰는 순환 방식으로 동작함.

- WAL 은 삽입, 업데이트 및 삭제를 포함해 데이터에 대한 변경 사항을 간결하고 효율적인 형식으로 기록.

5. Storage Manager

개념:

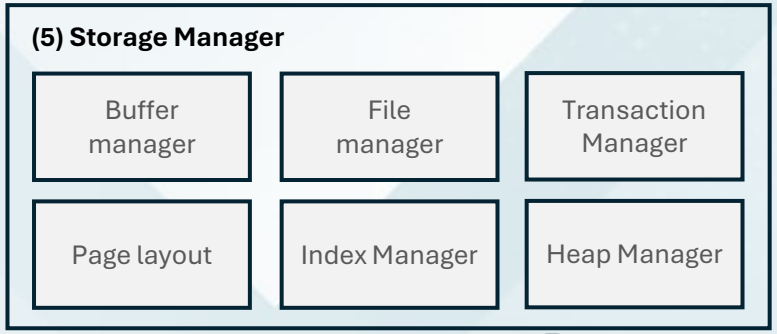
- Storage Manager는 PostgreSQL의 저장소 관리 계층으로, 데이터를 디스크에 체계적으로 저장하고 접근하는 역할을 함.
- PostgreSQL의 모든 데이터는 운영체제의 파일 형태로 저장되며, Storage Manager는 데이터 파일을 생성하고 관리하며, 필요한 페이지를 디스크에서 읽거나 메모리에 있는 변경 내용을 디스크에 기록함.



5. Storage Manager

Storage Manager 종류

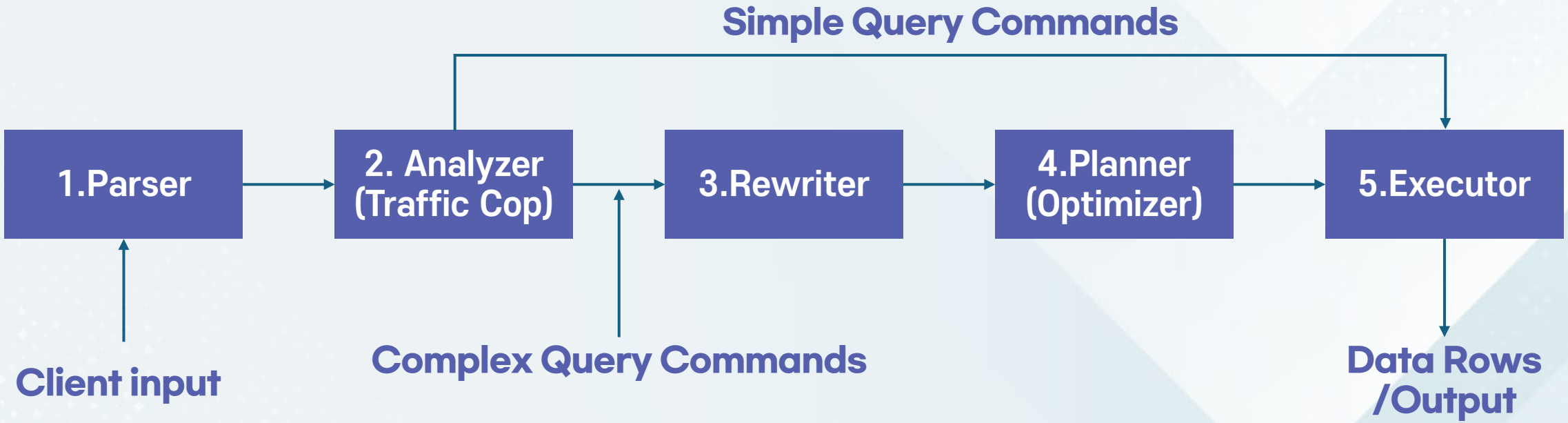
- **Buffer Manager:**
  - 최근에 액세스한 디스크 페이지의 메모리 내 캐시 관리 담당
  - 자주 액세스하는 페이지를 메모리에 캐싱해 성능을 향상시키고, 디스크와 메모리 간의 페이지 이동관리
- **File Manager:**
  - 데이터베이스를 구성하는 실제 파일관리 담당
  - 파일을 만들고 열고 닫으며 데이터베이스 개체에 대한 공간할당
- **Index Manager:**
  - 필요에 따라 인덱스를 생성, 업데이트 및 삭제하고 인덱스가 사용하는 디스크의 공간을 관리
- **Transaction Manager:**
  - 데이터베이스에 대한 변경 사항이 지속적이고 일관된 방식으로 기록되도록 함.
- **Page Layout:**
  - 디스크에 저장된 데이터의 구조와 형식을 정의
- **Heap Manager:**
  - 필요에 따라 테이블 내 공간을 할당 및 해제하고, 테이블 내 데이터 구성을 관리



클라우드 데이터베이스 Cloud Database

SQL 처리과정

SQL 처리과정


Simple Query Commands
1.Parser
2. Analyzer (Traffic Cop)
3.Rewriter
4.Planner (Optimizer)
5.Executor
Client input
Complex Query Commands
Data Rows /Output

SQL 처리과정

파서(Parsing 단계)

- PostgreSQL 의 파서는 질의 문자열을 입력받아 구문을 분석합니다. 질의가 SQL 문법에 맞는지 확인한 후, SELECT ... FROM ... WHERE ... 구조를 이해하고, 이를 파스 트리 형태로 변환함.

- 예를 들어, SELECT에 나열된 컬럼, 테이블, 필터 조건 등이 트리 구조로 분해됨

- 이 과정에서 테이블과 컬럼의 존재 여부, 데이터 타입과 연산자의 일치 여부 등도 검토됨. 이러한 검사를 통해 질의 구조를 나타내는 파스 트리가 생성되면, 다음 단계로 넘어감.

SQL 처리과정

분석기 (Traffic Cop)

- SQL 쿼리의 복잡성에 따라 간단한 경우에는 직접 실행기로 전달하고, 복잡한 경우에는 파스 트리를 통해 의미 분석을 수행하여 쿼리 트리를 생성함.

- 이 과정에서 의미 분석은 SQL이 참조하는 테이블, 함수 및 연산자를 이해하는 것이며, 존재하지 않는 테이블이나 열이 입력되었는지, 두 테이블 간의 조인이 불가능한지와 같은 의미적 오류가 없는지를 확인함.

SQL 처리과정

재작성 단계(rewriter)

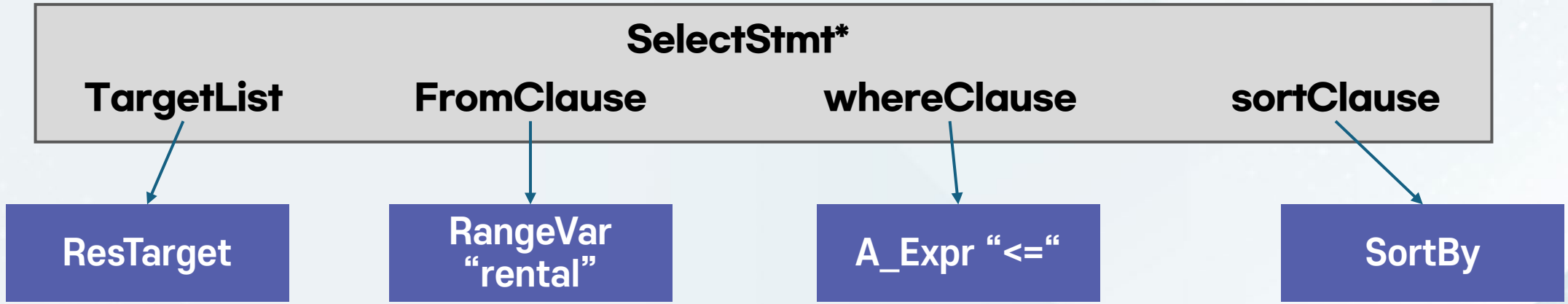
- 파싱된 질의 트리는 재작성 시스템(Rewrite System)을 통과함.

- 이 과정에서는 뷰(view)나 사용자 정의 룰(rule)이 질의에 적용되었는지 확인하고, 적용된 경우 해당 규칙에 따라 질의 트리를 변환함.

- 예를 들어, 질의 대상이 뷰일 경우 뷰 정의에 따라 기본 테이블 질의로 변환됨.
- 본 예시에서는 뷰나 특별한 룰이 없으므로, 재작성 단계에서 질의는 거의 그대로 유지됨.

SQL 처리과정

- SELECT customer_id, rental_rate from rental
  where rental_rate >=3.0
  order by customer_id



SQL 처리과정

플래너/최적화 단계

- 질의 트리(query tree)를 입력으로 하여 가능한 실행 방법을 탐색함.

- 예를 들어, film 테이블에서 Rating = 'R' 조건으로 title과 description을 선택할 때, 전체 테이블을 읽는 시퀀셜 스캔이나 인덱스가 있는 경우 인덱스 스캔을 통해 필요한 행을 찾는 방법이 있슴.

- 플래너는 다양한 실행 계획을 생성하고 각 계획의 비용(cost)을 예측하며, PostgreSQL은 통계 정보와 인덱스 유무를 바탕으로 가장 효율적인 계획을 선택함. 이 과정을 최적화 단계라고 하며, 최종 실행 계획은 다음 단계로 전달됨.

SQL 처리과정

플래너/최적화 단계에서 최적화기가 고려하는 사항

고려사항	설명
사용 가능 인덱스	쿼리의 테이블에 사용할 수 있는 인덱스를 검사하고 실행 계획에 사용할 가장 적합한 인덱스 선택
테이블 크기	쿼리와 관련된 테이블의 크기를 고려하고, 사용할 적절한 조인(JOIN) 알고리즘 선택
데이터에 대한 통계	쿼리의 다양한 조건의 선택성을 추정하기 위해 열의 유일한 값(distinct value)의 개수와 같은 데이터에 대한 통계 사용
구성설정	데이터 캐싱에 사용할 수 있는 메모리 양과 같이 데이터베이스 서버의 구성 설정을 고려하고, 이런 설정을 활용하는 실행 계획 선택

SQL 처리과정

Executor(실행 단계)

- 실행기(Executor)는 확보된 실행 계획을 받아 실제로 데이터베이스를 액세스함.

- 계획에 명시된 순서대로 테이블이나 인덱스를 탐색하여 조건을 만족하는 데이터를 읽어오고, 필요한 연산(필터 적용, 정렬나 집계 등이 포함되었다면 그런 연산)을 수행함.

연산 및 절 우선순위

SQL문을 수행할 때 연산 순서는 다음과 같음

1. 괄호( )
2. 곱셈, 나눗셈, 모듈러스(%, 나머지 반환) 산술 연산자
3. 더하기, 빼기 산술 연산자
4. 비교 연산자
5. 논리 연산자

절의 우선 순위

절	작성 우선순위	실행 우선순위
SELECT	1	6
FROM	2	1
JOIN	3	2
WHERE	4	3
GROUP BY	5	4
HAVING	6	5
ORDER BY	7	7
LIMIT	8	8

SUMMARY

- PostgreSQL의 아키텍쳐는 클라이언트 프로세스, 서버 프로세스, 시스템 메모리, 유틸리티 프로세스, 저장소 관리자로 구성된다.
- 클라이언트 프로세스는 데이터베이스에 질의를 보내고 결과를 수신하는 역할을 한다.
- 서버 프로세스는 SQL 문을 실제로 실행하고 트랜잭션을 관리한다.
- 시스템 메모리는 공유 메모리와 로컬 메모리로 나뉘며, 성능 향상을 위해 데이터를 캐싱한다.
- 유틸리티 프로세스는 데이터베이스의 일관성과 성능을 유지하기 위한 관리 작업을 수행한다.
- SQL 문은 5 단계를 거치면서 비용을 최적화 하는 플랜을 선택하여 실행된다.

클라우드 데이터베이스

Cloud Database

Thank you